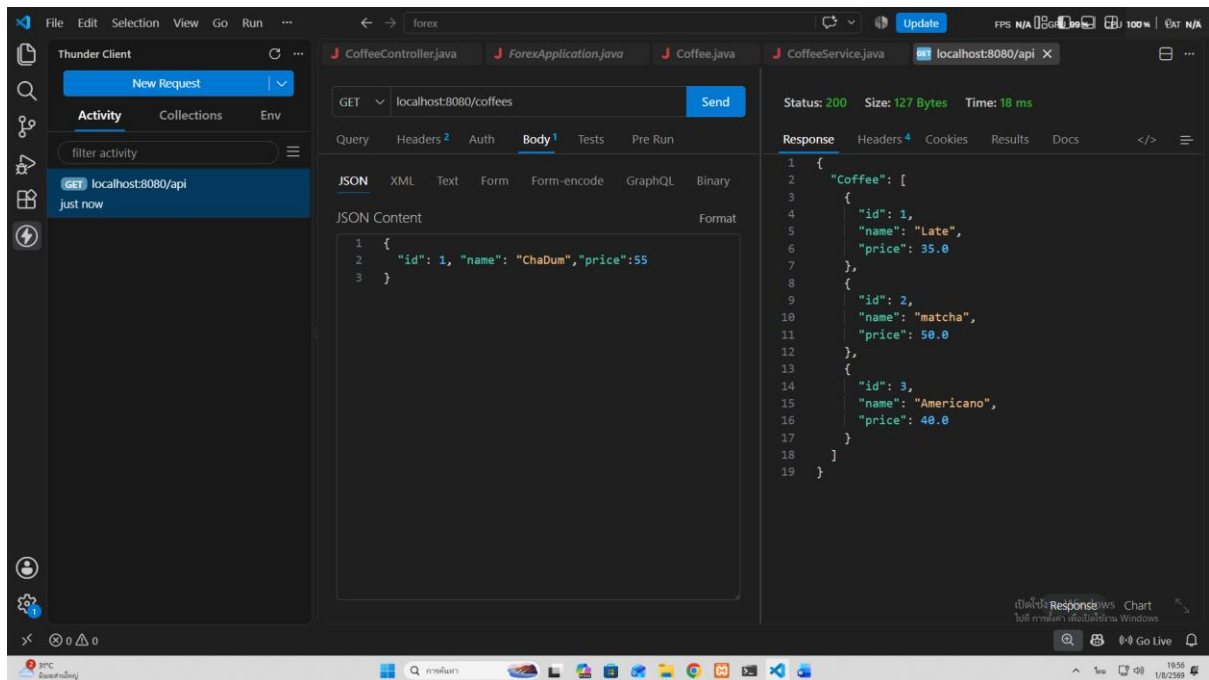
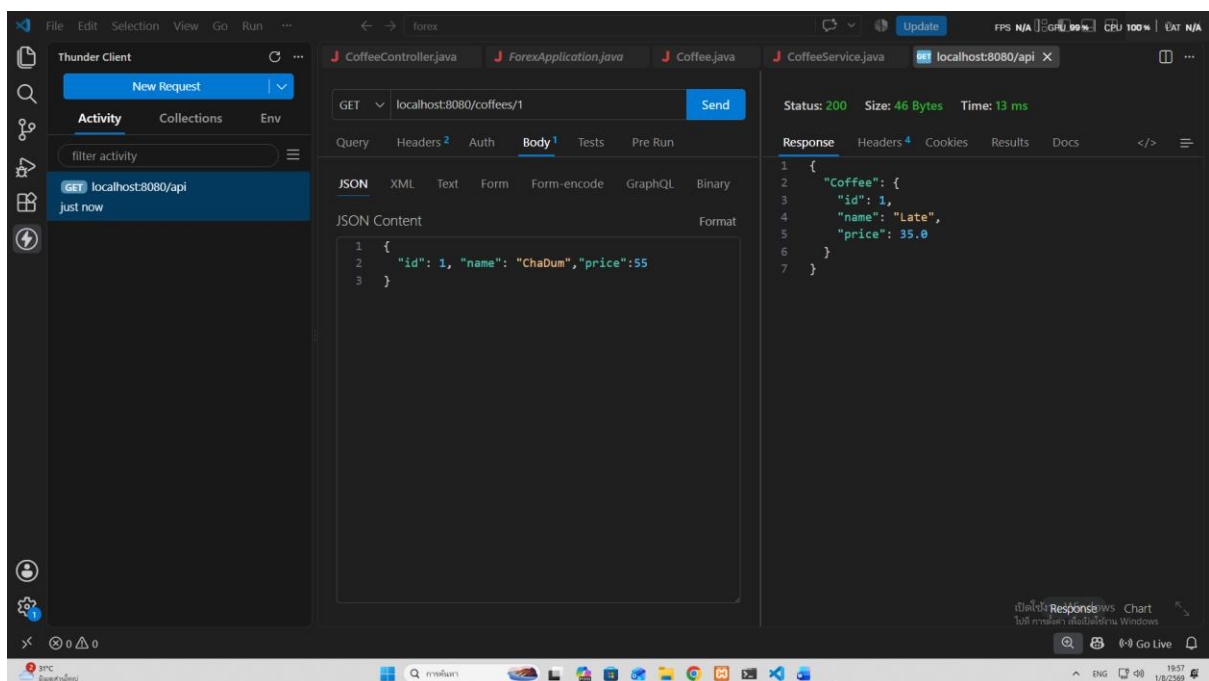


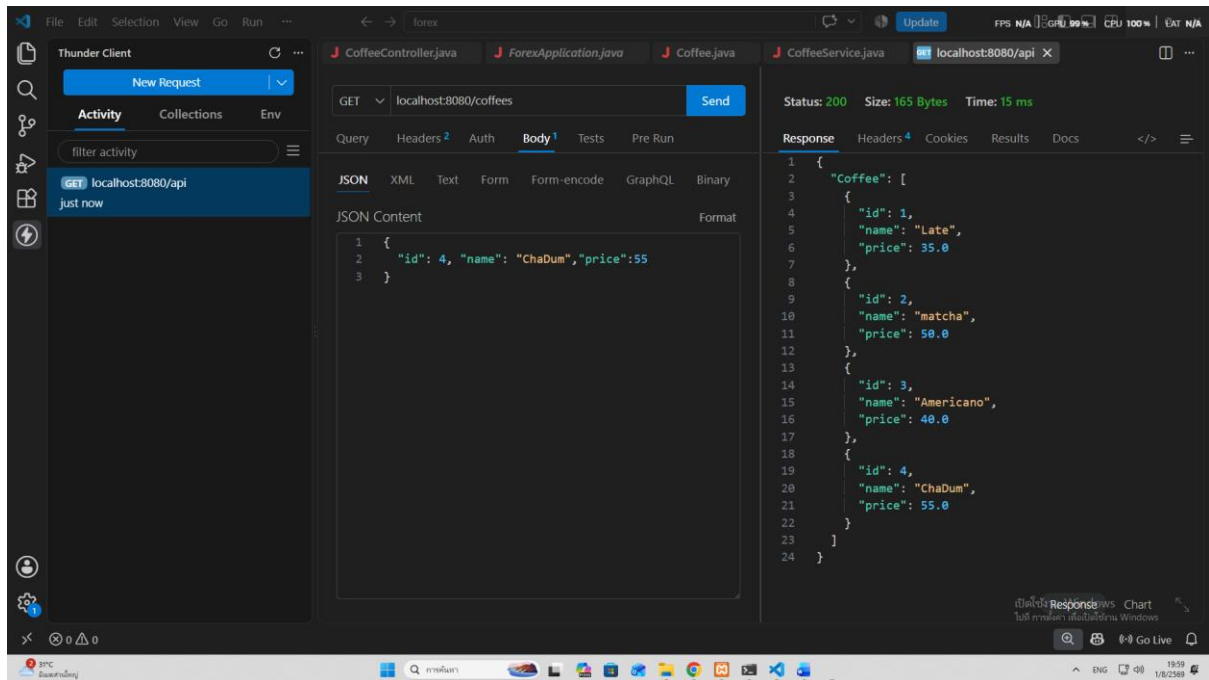
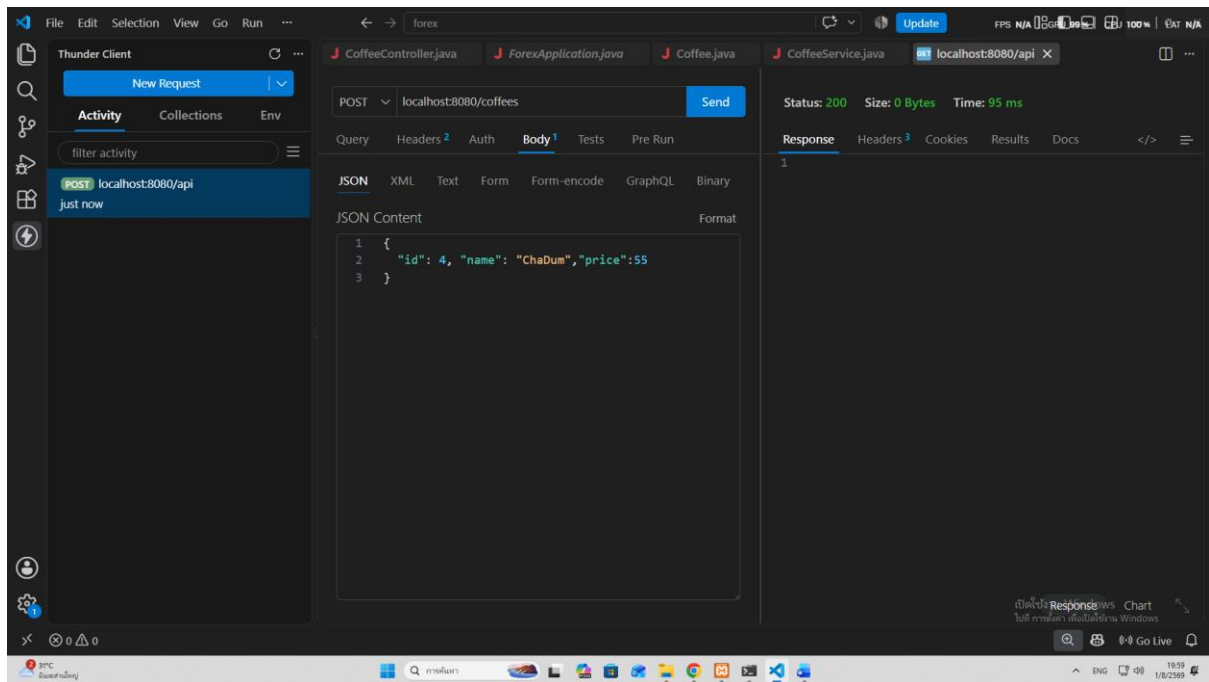
1.GetAll



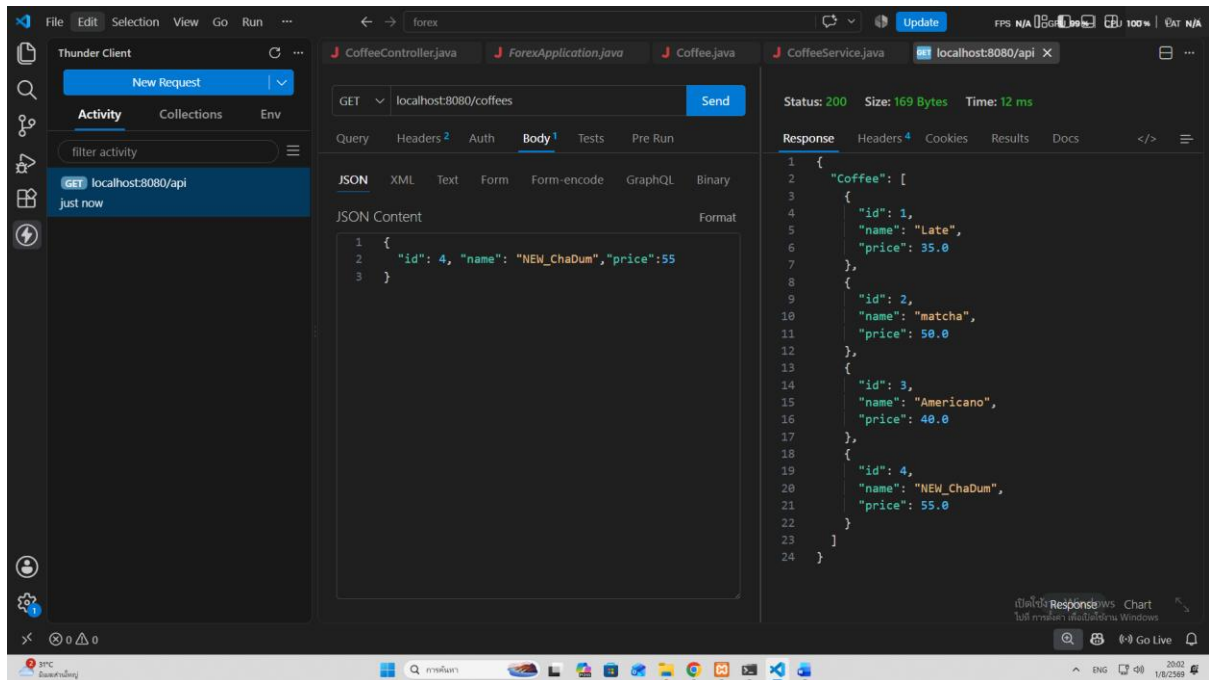
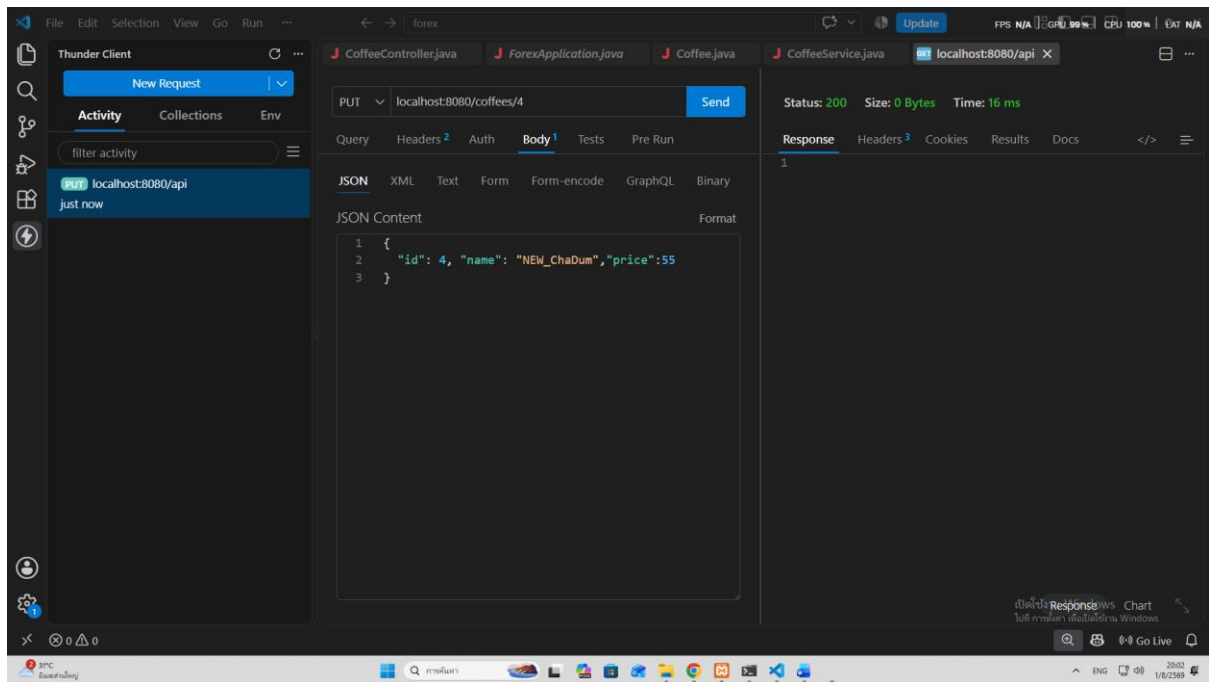
2.Get 1 menu



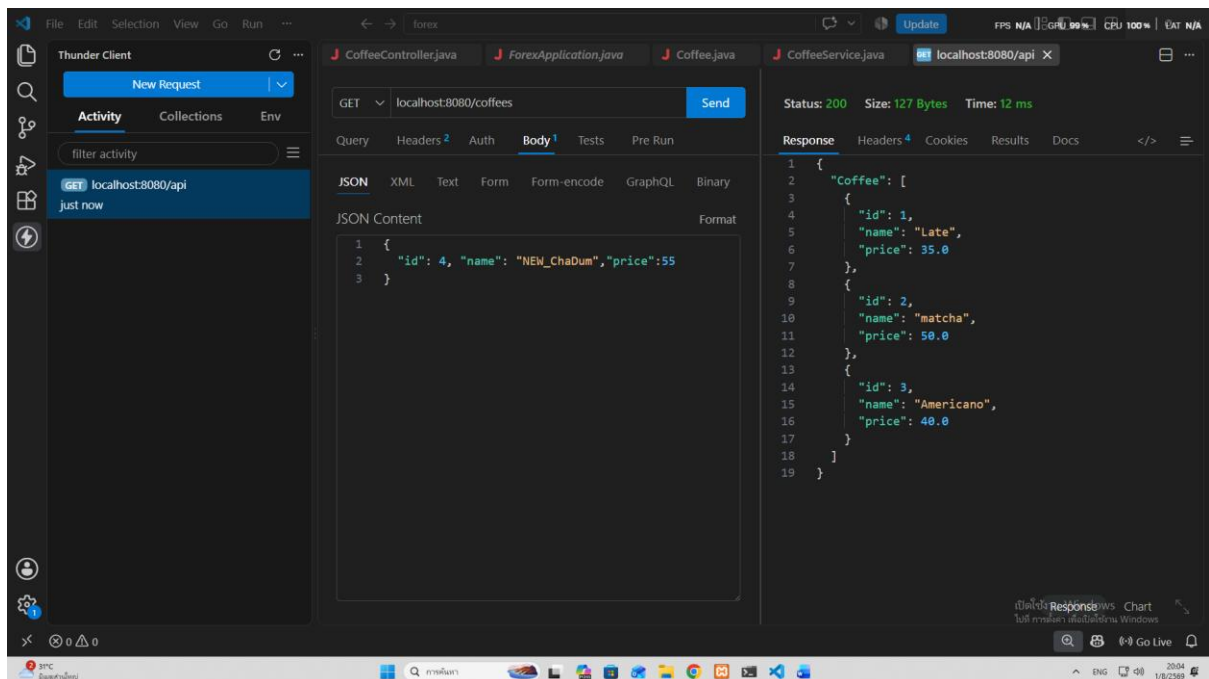
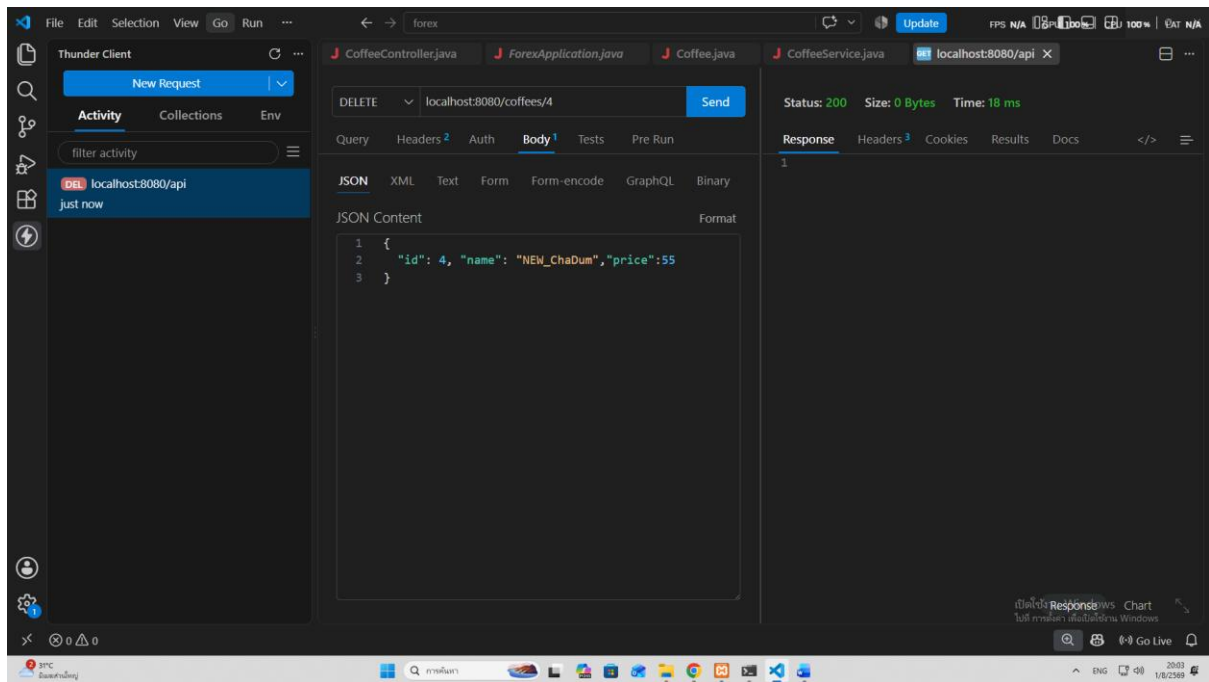
3.Post add new menu



4.PUT update menu



5.Delete Menu



Discussion

1. HTTP Method แต่ละตัว (GET / POST / PUT / DELETE) ต่างกันอย่างไร พร้อมยกตัวอย่างจากโปรเจกต์

HTTP Method เป็นวิธีที่ Client ใช้สื่อสารกับ Server โดยแต่ละ Method มีหน้าที่แตกต่างกัน

- **GET** ใช้สำหรับดึงข้อมูล โดยไม่เปลี่ยนแปลงข้อมูลในระบบ เช่น ในโปรเจกต์ Coffee API ใช้ GET `/coffee` เพื่อดึงรายการกาแฟทั้งหมด หรือ GET `/coffee/1` เพื่อดูข้อมูลกาแฟรหัส 1

- **POST** ใช้สำหรับสร้างข้อมูลใหม่ เช่น POST /coffee โดยส่งข้อมูลกาแฟผ่าน Request Body เพื่อเพิ่มเมนูใหม่เข้าไปใน List
- **PUT** ใช้สำหรับแก้ไขข้อมูลที่มีอยู่แล้ว เช่น PUT /coffee/1 เพื่อแก้ไขชื่อหรือราคาของกาแฟรหัส 1
- **DELETE** ใช้สำหรับลบข้อมูล เช่น DELETE /coffee/1 เพื่อลบกาแฟรหัส 1 ออกจากระบบ

กล่าวโดยสรุปคือ GET = อ่านข้อมูล, POST = เพิ่มข้อมูล, PUT = แก้ไขข้อมูล และ DELETE = ลบข้อมูล

2. ทำไมต้องแยก **Controller** กับ **Service** ออกจากกัน มีข้อดีอย่างไรถ้าโปรแกรมโตขึ้น

การแยก **Controller** และ **Service** เป็นหลักการ **Separation of Concerns** หรือการแบ่งหน้าที่ของแต่ละส่วนให้ชัดเจน

- **Controller** ทำหน้าที่รับ Request จาก Client และส่ง Response กลับไป
- **Service** ทำหน้าที่ประมวลผล Business Logic เช่น ค้นหาข้อมูล เพิ่มข้อมูล หรือคำนวณต่าง ๆ

ข้อดีเมื่อโปรแกรมมีขนาดใหญ่ ได้แก่

- โค้ดอ่านง่ายและเป็นระเบียบ
- แก้ไขหรือเพิ่มฟีเจอร์ได้ง่าย
- สามารถนำ Business Logic ไปใช้ซ้ำได้หลาย Controller
- ทดสอบ (Unit Test) ได้ง่ายกว่า
- หากเปลี่ยนฐานข้อมูล จะกระทบเฉพาะ Service ไม่ต้องแก้ Controller จำนวนมาก

ดังนั้น **Controller** ควรทำหน้าที่รับ-ส่งข้อมูล ส่วน **Service** ควรทำหน้าที่จัดการตรรกะของระบบ

3. ข้อมูลที่เก็บไว้ใน List ใน Memory หายไปตอนไหน และถ้าอยากให้ไม่หายควรทำอย่างไร

ข้อมูลที่เก็บอยู่ใน List จะถูกเก็บไว้ใน **RAM (Memory)**

ข้อมูลจะหายเมื่อ

- ปิดโปรแกรม
- Restart Application
- Server Restart
- Deploy โปรแกรมใหม่

เนื่องจากข้อมูลไม่ได้ถูกบันทึกลงใน **Storage** ถาวร

หากต้องการให้ข้อมูลไม่หาย ควรเก็บไว้ในฐานข้อมูล (Database) เช่น

- MySQL
- PostgreSQL
- MongoDB

หรืออาจบันทึกลงไฟล์ เช่น JSON หรือ CSV สำหรับระบบขนาดเล็ก

แนวคิดคือเปลี่ยนจากการเก็บข้อมูลใน **Memory** ไปเก็บใน **Persistent Storage** เพื่อให้ข้อมูลยังคงอยู่แม้โปรแกรมจะปิดหรือรีสตาร์ท

4. @RestController, @GetMapping, @PostMapping, @PathVariable, @RequestBody แต่ละตัวทำหน้าที่อะไร

- **@RestController** ใช้กำหนดว่าคลาสนี้เป็น REST API สำหรับรับคำขอจาก Client และส่งข้อมูลกลับในรูปแบบ JSON
- **@GetMapping** ใช้กำหนดว่าเมธอดจะทำงานเมื่อมีการส่งคำขอแบบ GET เพื่ออ่านหรือดึงข้อมูล
- **@PostMapping** ใช้กำหนดว่าเมธอดจะทำงานเมื่อมีการส่งคำขอแบบ POST เพื่อเพิ่มข้อมูลใหม่
- **@PathVariable** ใช้รับค่าที่อยู่ใน URL เช่น รหัสของข้อมูลที่ต้องการค้นหาหรือลบ
- **@RequestBody** ใช้รับข้อมูลที่ส่งมาจาก Client ผ่าน Request Body และแปลงเป็นอ็อบเจกต์ของ Java เพื่อให้สามารถนำไปประมวลผลต่อได้